

Multithreading in Python

Как добиться concurrency

- **multithreading** – несколько потоков внутри одного процесса
- `async/await` (event loop)

Concurrency. Multithreading

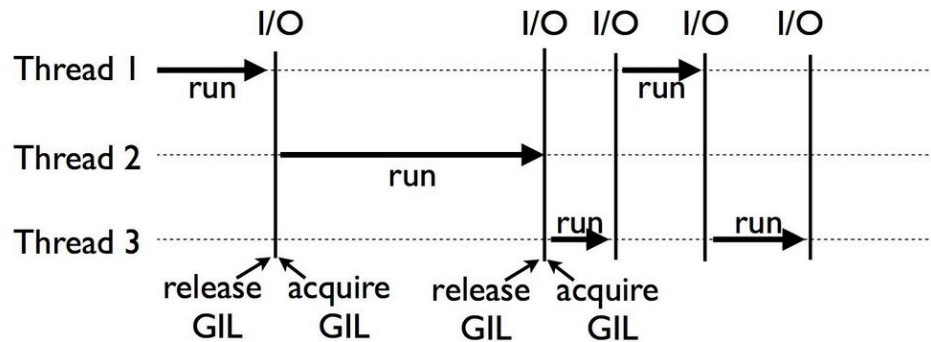
Multithreading – создание нескольких потоков (в рамках одного процесса)

Python — интерпретируемый язык, самая распространённая реализация — **CPython**. В CPython есть **GIL (Global Interpreter Lock)**: в один момент времени байткод Python выполняет только один поток (на уровне одного процесса).

Поэтому multithreading в чистом python не ускоряют CPU-bound код: даже на многоядерном CPU потоки будут по очереди владеть GIL.

Но multithreading может быть полезен для **I/O-bound** задач (сеть, диск, ожидание ответа API и т.д.):

- поток уходит ждать I/O – блокируется
- в этот момент GIL освобождается, и другие потоки могут исполнять python-код



Тогда можно сказать, что multithreading в python ускоряет не выполнение, а ожидание

Daemon-поток/процесс

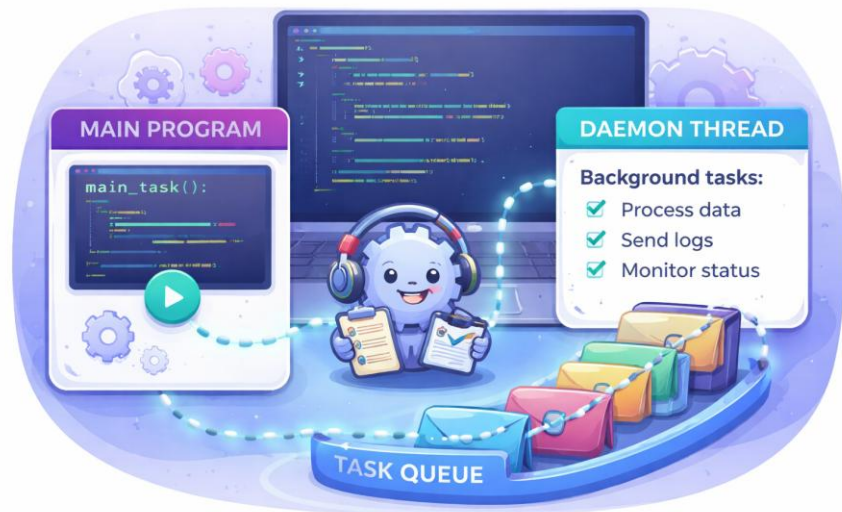
Daemon-поток/процесс – поток/процесс, который не мешает завершению основного потока/процесса

- Может быть полезен в реализации фоновых задач, которые существуют только пока живёт процесс и могут быть убиты в любой момент, то есть нам необязательно корректное завершение этого процесса

Формально мы можем даже не выставлять `daemon = True`, но концептуально (концепт фонового процесса) это может быть демон

Примеры:

- сервисный сбор статистик
- логирование в нескольких потоках, заводим отдельный поток-демон с очередью для синхронизации записей в общий файл
- демон-процесс, который собирает запросы в батч, которые затем идут в модель



Проблемы многопоточности

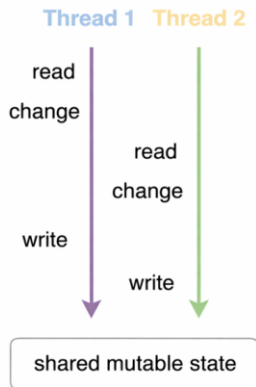
Race condition (состояние гонки)

Ключевая идея: несколько потоков **одновременно** работают с общими данными без синхронизации

Критическая секция — это участок кода, который работает с общим ресурсом (*shared mutable state*): переменная, БД, файл и тд

В результате **race condition** поведение становится недетерминированным

Race condition



Mutex

Mutex — это объект синхронизации (объект), который гарантирует, что в один момент времени **только один поток** может находиться в критической секции

Что делает mutex

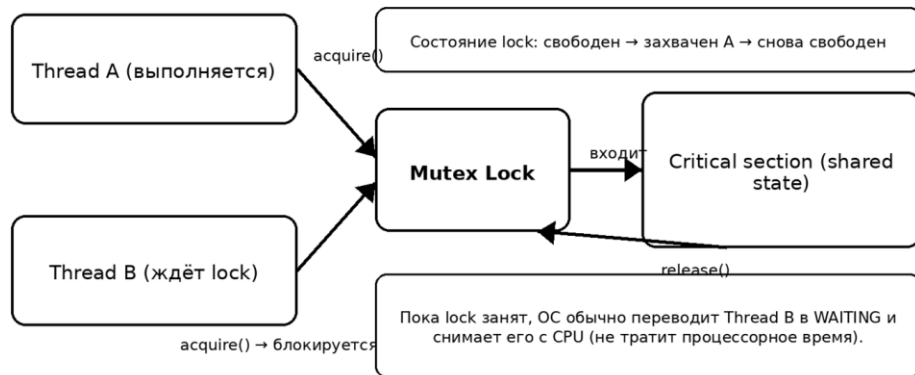
- защищает критическую секцию
- даёт доступ по одному потоку за раз

Предоставляет как минимум два обязательных метода:

- **acquire()** — попытаться захватить mutex (войти в критическую секцию)
- **release()** — освободить mutex (выйти из критической секции)

Как работает mutex (Lock)

Один поток в критической секции, остальные ждут



Lock, RLock

В python mutex представлен классом **Lock** – обвязка поверх mutex внутри ОС

Lock — это простой mutex без понятия владельца – может находиться в двух состояниях **free / locked**:

```
from threading import Lock  
lock = Lock()
```

- **acquire()** — захватить (если уже locked – поток ждёт/блокируется)
- **release()** — освободить (владельца не проверяет)

Свойства

- не знает, какой поток его захватил
- нельзя захватить повторно (**self-deadlock**)
- быстрее, чем RLock

RLock — re-entrant Lock (реентерабельный)

```
from threading import RLock  
rlock = RLock()
```

Дополнительное состояние

- **owner_thread_id** – поток-владелец
- **recursion_count** – счетчик того, сколько раз поток-владелец захватил lock

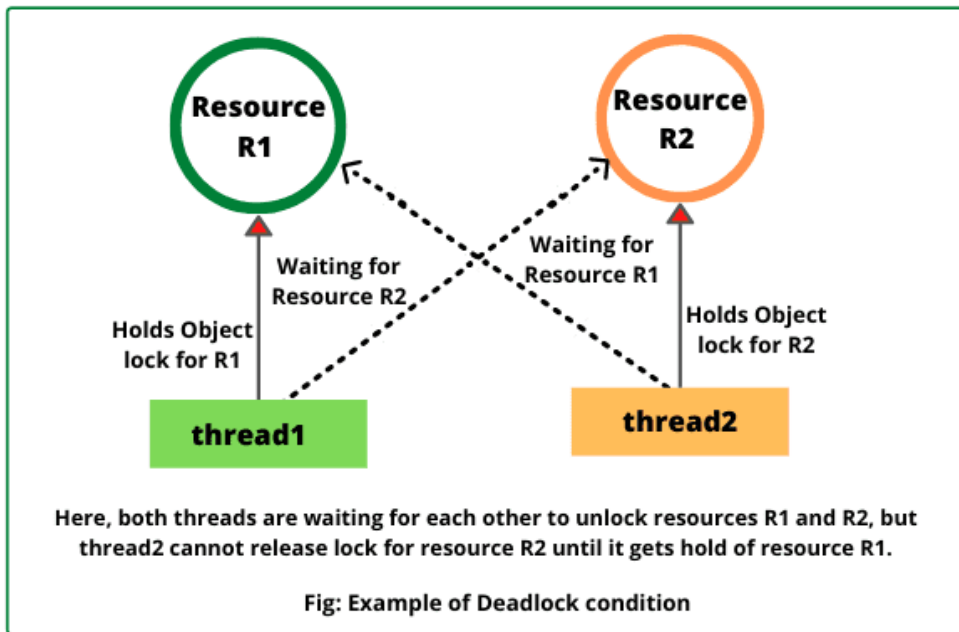
Свойства

- поток может входить в lock несколько раз
- **другой поток не может сделать release чужого lock**
- предотвращает self-deadlock

Deadlock

Deadlock (взаимная блокировка) — состояние системы, при котором набор потоков/процессов заблокирован навсегда, потому что каждый из них ждёт ресурс/событие, которое может быть предоставлено только другим участником этого набора, но тот тоже ждёт

В качестве “ресурса” может быть, например, mutex или Lock/Rlock



Другие проблемы синхронизации

- TOCTOU как частный случай гонки (time-of-check/time-of-use) – <https://habr.com/ru/articles/764234/>
- Starvation на примере «философов» (Go) – <https://habr.com/ru/articles/951224/>
- Livelock (ожившая блокировка) – <https://habr.com/ru/articles/966892/>

Пример (thread-safe *least-recent used* cache)

```
@lru_cache(maxsize=16)
```

```
def some_method(self, *args, **kwargs):  
    pass
```

```
instance.cache = OrderedDict()  
self.lock = Rlock()
```

1-й Lock:

- корректно изменяет shared-объект, т.к. иначе может нарушиться порядок приоритетности
- корректно обновляет счетчик

2-й Lock:

- предотвращает переполнение кеша
- предотвращает лишнее вытеснение
- корректно обновляет счетчик

```
instance_cache = instance.cache[self.cached_attr]  
result = instance_cache.get(key, self.default)
```

```
if result is not self.default:  
    with self.lock:  
        instance_cache.move_to_end(key)  
        self.stats[instance_hash]['hit'] += 1  
        return copy(result) if copy_on_return else result
```

```
# The result was not found in cache: evaluate function  
result = func(*args, **kwargs)
```

```
# Add the result to cache  
with self.lock:  
    self.stats[instance_hash]['miss'] += 1
```

```
if key in instance_cache:  
    pass  
elif len(instance_cache) >= self.maxsize:  
    instance_cache.popitem(last=False)  
    instance_cache[key] = result  
else:  
    instance_cache[key] = result
```

```
return copy(result) if copy_on_return else result
```

Semaphore (семафор)

Semaphore — объект синхронизации, который управляет внутренним счетчиком разрешений к общему ресурсу

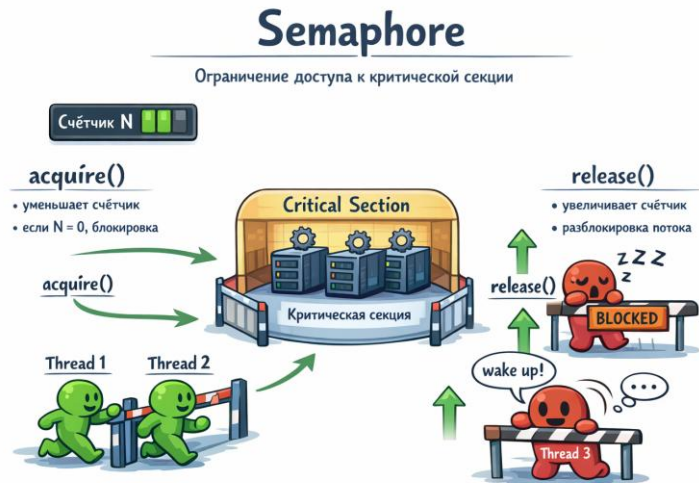
Принцип работы:

- есть **счётчик N** (количество permits).
- **acquire()** – уменьшает счётчик; если счётчик = 0 – поток блокируется
- **release()** – увеличивает счётчик и может разбудить ожидающего

Применение

- Ограничение количества одновременных операций: API с лимитом запросов
- Используется в очереди (Queue) как ограничитель количества задач, входящих в очередь

```
from threading import Semaphore  
sem = Semaphore(value=2)
```



ThreadPoolExecutor

- Высокоуровневая абстракция над `threading`: пул рабочих потоков
- Когда есть много похожих задач, которые нужно запускать “параллельно”
- Обычно удобнее и безопаснее, чем ручное управление потоками
- Подходит для ускорения на I/O-bound задачах

Что внутри:

- Сам создаёт и переиспользует потоки (**workers**) и распределяет задачи (**tasks**) по worker-потокам
- Создает очередь для управления задачами
- Создает объект **future**, служащий контейнером для результата
- Корректно закрывает все потоки из пула – потокобезопасен относительно **submit** и **shutdown** (важно, когда PoolExecutor вызывают несколько потоков)

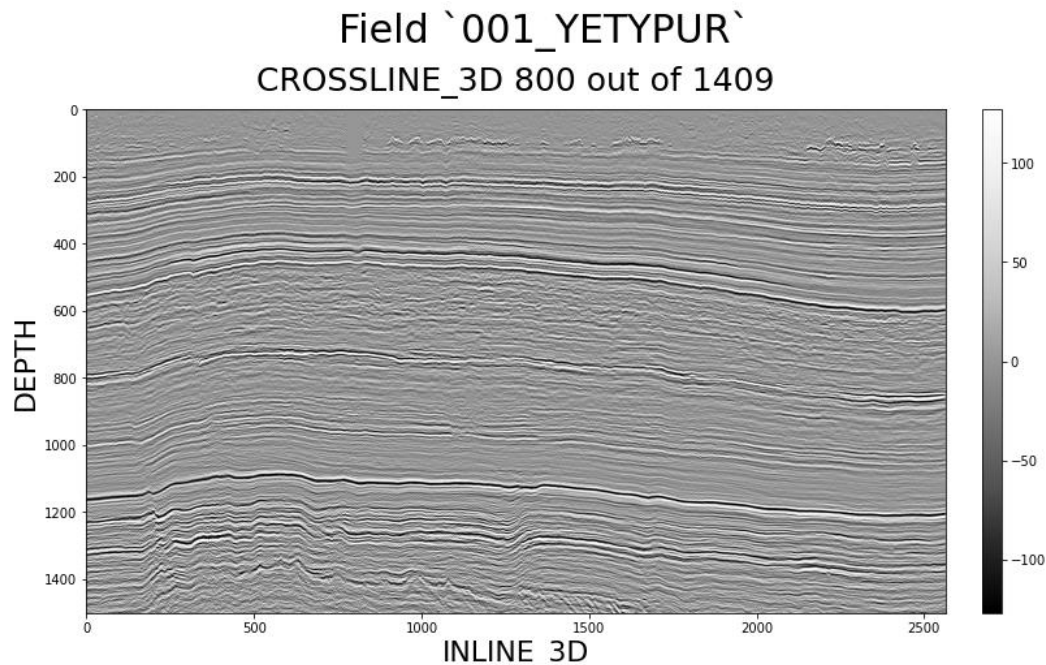
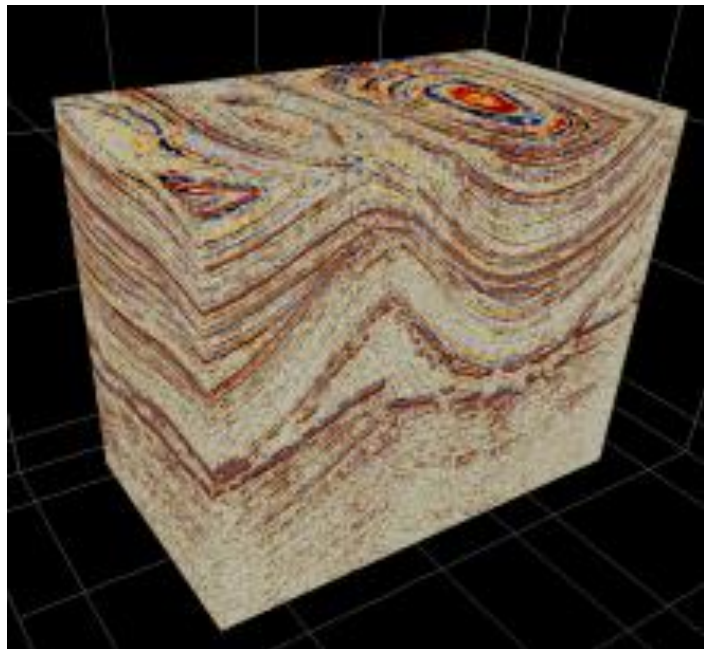
```
from concurrent.futures import ThreadPoolExecutor
import time
```

```
def task(n):
    time.sleep(1)
    return n
```

```
max_workers = 3
with ThreadPoolExecutor(max_workers=max_workers) as executor:
    futures = [executor.submit(task, n) for n in range(5)]
    results = [f.result() for f in futures]
```

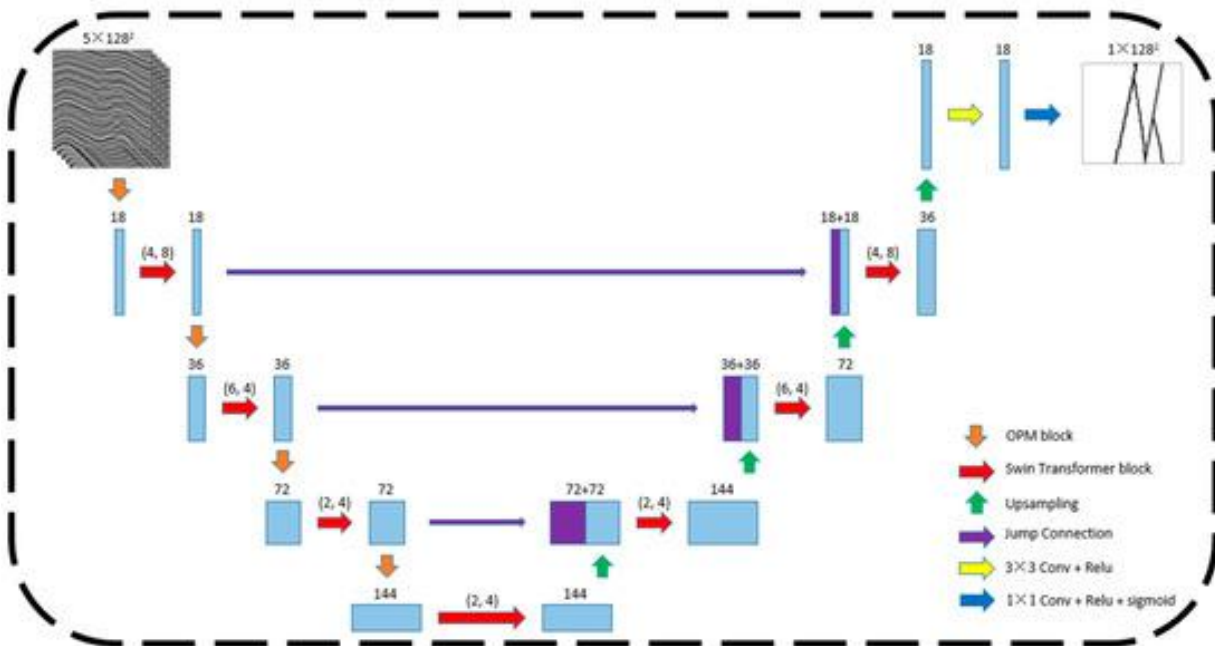
Пример `load_seismic`

- 2D locations – `(1, i/x, d)` – набор `slice()` по каждой оси
- загрузка кропов происходит через чтение из **SEG-Y** файла, представляющего собой бинарник определенной структуры (заголовки + **трассы**)
- доступ к трассам через `mmap`: не загружаем разом весь куб, а грузим трассы по требованию – **I/O-bound**



Пример `load_seismic`

- 2.5D – хотим грузить несколько каналов в кропе –
(`channel_number`, `i/x`, `depth`)
- Чем больше `channel_number`, тем ощутимей будет ускорение от использования нескольких потоков
- Ощутимее всего ускорение будет при загрузке 3D кропов



Пример `load_seismic`

```
locations_with_adjacent = np.array([
    [..., 80, 10, 20, 81, 138, 148],
    [..., 100, 10, 20, 101, 138, 148],
    [..., 120, 10, 20, 121, 138, 148]
])
```

```
def load_seismic_orientation_0(field, locations, src_geometry, buffer, **kwargs):
    _ = field.load_seismic(locations=locations, src=src_geometry, buffer=buffer, **kwargs)
```

```
def load_seismic_orientation_1(field, locations, src_geometry, buffer, ix, i, **kwargs):
    data = field.load_seismic(locations=locations, src=src_geometry, **kwargs)
    buffer[ix, i] = data.transpose(1, 0, 2)
```

```
field.load_seismic --> traces = self.data_mmap[indices, limits]
```


Пример `load_seismic`

```
iterator = enumerate(zip(locations with adjacent, orientations, field names))
```

```
for ix, (location with adjacent, orientation, field name) in iterator:
```

```
with AdaptivePoolExecutor(max_workers=max_workers) as executor:
```

```
field = fields[field name]
```

```
for i, location in enumerate(location with adjacent):
```

```
if orientation == 0:
```

```
_ = executor.submit(load_seismic_orientation_0,
                    field=field,
                    locations=location,
                    src_geometry=src_geometry,
                    buffer=buffer[ix, i][None, :], **kwargs)
```

```
elif orientation == 1:
```

```

_ = executor.submit(load_seismic_orientation_1,
                    field=field,
                    locations=location,
                    src_geometry=src_geometry,
                    buffer=buffer,

```

Пример `load_seismic`

```
iterator = enumerate(zip(locations with adjacent, orientations, field names))
```

```
with AdaptivePoolExecutor(max_workers=max_workers) as executor:
```

```
for ix, (location_with_adjacent, orientation, field_name) in iterator:
    field = fields[field_name]
```

```

for i, location in enumerate(location_with_adjacent):
    if orientation == 0:
        _ = executor.submit(load_seismic_orientation_0,
                             field=field,
                             locations=location,
                             src_geometry=src_geometry,
                             buffer=buffer[ix, i][None, :], **kwargs)
    elif orientation == 1:
        _ = executor.submit(load_seismic_orientation_1,
                             field=field,
                             locations=location,
                             src_geometry=src_geometry,
                             buffer=buffer,
                             ix=ix, i=i, **kwargs)

```